AI-Driven Level 2 Driver Assistance for Electric Vehicles

An undergraduate project progress report submitted to the

Department of Computer Engineering
Faculty of Engineering
University of Ruhuna
Sri Lanka

in partial fulfillment of the requirements for the

Degree of the Bachelor of the Science of Engineering Honours

by

Alagiyawanna A.M.A.K - EG/2021/4395
Bandara R. M. U. T - EG/2021/4437
Fernando K.M.K.N.C - EG/2021/4508
Jackshan Venujan G.S - EG/2021/4566

.....
Dr. Kushan Sudheera
(Supervisor)

Abstract

This project proposes the development of a Level 2 Advanced Driver Assistance Systems for an electric vehicle. In furtherance, the proposed system is designed to facilitate ease of transition from Level 1 to Level 2 autonomy by enhancing core perception, decision-making, and control functionalities while maintaining human supervision in accordance with Level 2 operational standards. The main development tasks include lane keeping assistant, adaptive cruise control, decision making, and trajectory planning, and improvement of vision perception capabilities, such as real-time object detection, lane detection, and traffic signal recognition. Object detection briefly runs for the detection of surrounding vehicles and pedestrians, lane detection for road geometry estimation, and detection of traffic lights to recognize signal states. An intelligent trajectory planner then utilizes the information for the generation of safe steering and acceleration commands. Lane Keeping Assistant controls lateral motion, while longitudinal motion is maintained through Adaptive Cruise Control to keep safe inter-vehicle distances. Herein, the system will adopt multimodal sensor fusion techniques that will include cameras and Light Detection and Ranging sensors, whose merge enhances robustness and raises environmental awareness. A decision-making framework optimizes vehicle behavior, trained in the CARLA simulation environment and adapted for real-world application through simulation-to-reality transfer techniques. The architecture follows a modular structure to improve scalability, facilitate debugging, and support iterative development. Validation encompasses simulation testing, hardware-in-the-loop testing, and controlled real-world trials, establishing a safe and reliable research framework for Level 2 autonomous vehicle development.

Keywords: Advanced Driver Assistance System, Electric Vehicles, Level 2 Autonomy, Reinforcement Learning, , Sensor Fusion

Contents

Acronyms **vii**

1 Introduction **1**
1.1 Problem Background 1
1.2 Problem Statement 2
1.2.1 Challenge 1: The Sim-to-Real Perception Gap 2
1.2.2 Challenge 2: Computational Constraints of Edge Deployment 2
1.2.3 Challenge 3: Architectural Modularity and Debugging . . . 2
1.2.4 Challenge 4: Intelligent Decision-Making in Complex Scenarios 3
1.2.5 Challenge 5: Real-Time Performance with Multi-Modal Sensors 3
1.2.6 Challenge 6: Hardware Integration and Actuation 3
1.3 Objectives and Scope 4
1.3.1 Objectives 4
1.3.2 Scope 4

2 Literature Review **7**
2.1 Previous Work 7
2.1.1 Autonomous Driving Levels and L2 Systems 7
2.1.2 System Architecture and Sensor Fusion Architectures 9
2.1.3 Sensor Technologies 9
2.1.4 Perception and Computer Vision 10
2.1.5 Path Planning and Decision Making 11
2.1.6 Control Systems 12
2.1.7 Level 2 Autonomous Systems Integration in Electric Vehicles 13
2.1.8 Safety and Validation 14
2.2 Gaps in Literature 16

3 Methodology **18**
3.1 Research Design 18
3.1.1 System Architecture 18
3.1.2 Simulation Environment Development 19
3.2 Light Detection and Ranging (LiDAR)-Perception Module Development and Experimentation 19
3.2.1 Research Design and Core Theory 20
3.2.2 Implementation of the LiDAR Perception Module 22
3.2.3 Lane Keeping Assistance Development 24
3.2.4 Trajectory Planning and Decision-Making Development . . . 24

3.2.5 Trajectory Planning Module 25
3.2.6 Adaptive Cruise Control Development 26

4 Testing and Performance Evaluation 28
4.1 Experimental Setup 28
4.1.1 Obstacle Identity Matching Across Sensors 28
4.1.2 KPIs and Metrics 29
4.2 Results and Analysis 31
4.2.1 Distance Accuracy 31
4.2.2 Processing Latency 31

5 Discussion and Insights 32
5.1 Summary of Key Results and Observations 32
5.2 Insights and Engineering Decisions 32
5.2.1 Objectives Achieved? 33
5.2.2 A Plausible Real-World Application 34
5.2.3 Limitations and Drawbacks 34
5.2.4 Concluding Remark 35

6 Timeline and Resource Required 36
6.1 Timeline 36
6.1.1 7^{th} Semester (December 2025 - April 2026) 36
6.1.2 8^{th} Semester (May 2026 - September 2026) 37
6.2 Resources Required 38

7 Conclusion 39

References 39

List of Figures

2.1 Multi-sensor coverage and functional mapping in a Level-2 Advanced Driver Assistance Systems (ADAS). 10

3.1 Hierarchical autonomous vehicle system architecture showing the four primary layers: Sensing, Perception, Planning, and Control. . . 18
3.2 LiDAR perception module architecture 22

List of Tables

3.1 LiDAR sensor configuration, from the system implementation. . . . 22

4.1 KPIs used in the camera-versus-LiDAR comparison. 30
4.2 Verification test cases in `test_obstacle_ids.py` and their significance. 30
4.3 Distance accuracy vs. CARLA ground truth (filtered `BOTH`-tagged rows). 31
4.4 Per-frame processing latency of the camera and LiDAR pipelines. . 31

5.1 Insights from the camera–LiDAR comparison and the decisions they drive. 33

6.1 Timeline of activities for the 7^{th} semester (December 2025 - April 2026) 36
6.2 Timeline of activities for the 8^{th} semester (May 2026 - September 2026) 37
6.3 Estimated hardware expenditure for the development and validation of the proposed ADAS system 38

Acronyms

ACC	Adaptive Cruise Control
ADAS	Advanced Driver Assistance Systems
AI	Artificial Intelligence
AV	Autonomous Vehicle
CAN	Controller Area Network
CNN	Convolutional Neural Network
DRL	Deep Reinforcement Learning
ECU	Electronic Control Unit
EV	Electric Vehicle
FSD	Full Self-Driving
FOV	Field of View
GPS	Global Positioning System
HIL	Hardware-in-the-Loop
HPC	High-Performance Computing
ISO	International Organization for Standardization
LCA	Lane Centring Assistance
LCC	Lane Centring Control
LiDAR	Light Detection and Ranging
LKA	Lane Keeping Assistant
ML	Machine Learning
MPC	Model Predictive Control
ODD	Operational Design Domain
PID	Proportional-Integral-Derivative
RADAR	Radio Detection and Ranging
RL	Reinforcement Learning
ROS	Robot Operating System
SAE	Society of Automotive Engineers
SOTIF	Safety of the Intended Functionality
TEB	Timed Elastic Band
TTC	Time to Collision
V2I	Vehicle-to-Infrastructure
V2V	Vehicle-to-Vehicle

Chapter 1

Introduction

1.1 Problem Background

The automotive industry is experiencing a paradigm shift towards automation, triggered by the prospect of lowering accident rates and improving traffic flow. Advanced Driver Assistance Systems (ADAS), particularly the automation standard for Level 2 (L2) as classified by the Society of Automotive Engineers (SAE), represent a fundamental development in this area. L2 automation provides sustained lateral (steering) and longitudinal (acceleration and braking) control while operating under constant human observation [1]. These technologies have already been developed in the automotive industry using many sensors and learning-based systems, as seen in self-driving cars from companies like Tesla and Waymo. However, there is still a large "Safety Technology Divide." Most existing systems are designed for well-structured roads and traffic conditions found in developed countries. As a result, they often do not work well in unstructured and unpredictable traffic environments commonly seen in developing regions. This proposed project builds directly on the successful results of Phase 1. In Phase 1, deep learning models for lane detection and object classification showed good performance on synthetic datasets. In addition, Proportional-Integral-Derivative (PID) algorithms were able to control the vehicle effectively in a simulated urban environment using the CARLA simulator. The proposed project now moves to Phase 2: Focusing on level 2 autonomy. In this phase, the work integrates the level 1 autonomy features along with level 2 features including lane keeping assistant, adaptive cruise control, decision making and trajectory planning. The main goal is to integrate computer vision and decision making algorithms to achieve level 2 autonomy. To achieve this, the project will use LiDAR sensors together with cameras. Unlike cameras, LiDAR sensors are not affected by changes in lighting or surface textures. They can provide accurate 3D distance and shape information, which is essential for detecting obstacles in real-world driving conditions. This phase will focus on reducing the simulation-to-real (Sim-to-Real) gap by addressing practical challenges such as sensor noise, changing lighting conditions. The final goal is to develop a working Level-2 driver assistance system that can be added to existing electric vehicles as a retrofit solution.

1.2 Problem Statement

Although current autonomous driving techniques show strong potential, there are technical issues regarding system-level implementation associated with reliability, real-time systems, and robustness.

1.2.1 Challenge 1: The Sim-to-Real Perception Gap

Problem: Testing a strong vision-based perception algorithm directly through real-world experiments is dangerous and expensive. On the other hand, models trained on more refined simulators (such as (like Car Learning to Act (CARLA))) can easily lead to serious deterioration in their performance when executed on real vehicles concerning the difference in illumination levels, textures, and noise present in the sensor (domain shift). A major challenge is the correct identification of detected road markings and objects on a real vehicle.

Our Approach: The system used a simulation-first but hardware-aware approach to development, in that it synchronized the simulation sensor configurations with the requirements of the camera and Light Detection and Ranging (LiDAR) hardware. Strategies such as domain randomization and validation of geometric consistency to ensure the robustness of perception are used to ensure a smooth transition from a simulation environment to the real world.

1.2.2 Challenge 2: Computational Constraints of Edge Deployment

Problem: Implementing sophisticated AI-driven algorithms for Object Detection, Lane detection, and Traffic Signal Recognition requires substantial computational power. Benchmarking complex neural networks on embedded hardware (NVIDIA Jetson Orin) often reveals latency issues that compromise safety at operational speeds of 10–30 km/h [2].

Our Approach: Computational efficiency is handled with light-weight perception models, task-dependent execution speed, and the use of classical planning and control methods. Reinforcement learning is limited to low-dimensional parameter updates that enable real-time execution on embedded systems.

1.2.3 Challenge 3: Architectural Modularity and Debugging

Problem: A monolithic end-to-end autonomous driving solution makes it very hard to break down where the errors occur. It is hard to know where the problem is occurring in perception, planning, or control.

Our Approach: The hierarchical and modular design with well-defined interfaces makes it easy to distinguish between perception, planning, control, and sen-

sor components. Thus, individual module testing, fault localization, and system integration become feasible.

1.2.4 Challenge 4: Intelligent Decision-Making in Complex Scenarios

Problem: Rule-based systems and supervised learning struggle with the infinite variety of real-world scenarios and high-dimensional urban environments. Traditional methods often become brittle when managing discrete decisions, such as yielding or handling red lights, in unpredictable settings [3].

Our Approach: A hybrid decision-making framework combines rule-based safety enforcement with reinforcement learning assisted parameter adaptation. Reinforcement learning modulates risk sensitivity and behavior preferences under varying traffic conditions without directly controlling vehicle maneuvers, preserving predictability and safety.

1.2.5 Challenge 5: Real-Time Performance with Multi-Modal Sensors

Problem: Single-sensor systems (camera-only) are unreliable in poor lighting, yet processing 3D LiDAR data is computationally heavy. Fusing these streams in real time for accurate depth estimation and obstacle avoidance creates a significant processing bottleneck [4].

Our Approach: Selective object-level fusion integrates camera-based semantic perception with LiDAR based geometric validation. This reduces computational load while maintaining reliable depth estimation and obstacle awareness in real time.

1.2.6 Challenge 6: Hardware Integration and Actuation

Problem: Control Interfacing high-level digital logic (Robot Operating System (ROS)2 commands) with the physical vehicle's drive-by-wire system requires precise low-level communication. Misinterpretation of signals or communication latency via the Controller Area Network (CAN) bus can lead to erratic vehicle behavior [5].

Our Approach: A structured control interface converts high-level planning outputs into constrained actuator commands. Rate limiting, safety bounds, and standardized messaging ensure smooth and reliable interaction with vehicle drive-by-wire systems.

1.3 Objectives and Scope

1.3.1 Objectives

In Phase II, the focus is to design and implementation of an autonomous driving assistance system targetting level 2 autonomy. The proposed system adopts a modular architecture incorporating AI-driven computer vision and decision-making algorithms. The specific objectives to be achieved are as follows:

- Enhance and integrate vision-based perception capabilities, including real-time object detection, lane detection, and traffic signal recognition from level 1 autonomy.
- Implement a Lane Keeping Assistance System to reliably detect lane boundaries and provide continuous lateral control support, enabling the vehicle to maintain stable lane centering under supervised Level 2 driving conditions.
- Develop an intelligent trajectory planning and decision-making system capable of generating safe and feasible speed and steering control commands under supervised driving conditions.
- Design and implement an adaptive cruise control module for regulating vehicle speed based on detected obstacles and traffic conditions.
- Test and validate the integrated Level 1 and Level 2 functionalities through simulation to ensure safe, reliable, and integrated system performance under supervised driving conditions.

1.3.2 Scope

The scope of this project is to develop the L2 Autonomous Driving Assistance System and integrate it into a real-world vehicle platform.

Perception System Development

- **Object Detection:** Detection of stationary and moving objects within operational range to enable safe navigation and obstacle avoidance in urban environments at 10–30 km/h.
- **Lane Detection and Keeping:** Identifying left and right lane boundaries, solid and dashed lane markings within operational range, assisting steering to maintain lane position, and adapting from simulation to real-world conditions.
- **Traffic Signal Recognition:** Able to recognize traffic light states and road sign boards to keep the retheme of the vehicle.
- **Sensor Integration:** Integration of Arducam Global Shutter cameras and LiDAR sensors on NVIDIA Jetson platform for enhanced environmental perception.

Control System Development

- **Trajectory Planning:** Develop a trajectory planning module that generates safe paths with the help of a perception system. Integrate from simulation to a real-world vehicle platform.
- **Decision-Making System:** Implementation of AI-driven algorithms processing perception inputs to determine vehicle actions for safe obstacle avoidance. Testing in the CARLA simulator before hardware integration.
- **Adaptive Cruise Control:** Speed regulation module adjusting velocity based on perception system outputs. Simulation testing in CARLA before real-world implementation.

System Integration and Testing

- **Simulation Testing:** Validation of new control modules in the CARLA simulation environment to verify functionality and safety before deployment.
- **Hardware Deployment:** Integration on NVIDIA Jetson platform with a complete sensor suite including camera and a LiDAR for field testing under controlled conditions.

Operational Constraints

- Operation limited to urban roads with clear lane markings.
- System functionality restricted to daylight driving conditions.
- Vehicle operating speed range: 10 - 30 km/h.
- Effective lane detection range: approximately 10 - 20 meters.
- Continuous driver supervision required with manual override available at all times.
- Testing conducted in controlled environments, not in uncontrolled public traffic.

Exclusions

The scope of this project excludes:

- Strategic maneuvers such as overtaking or complex intersection navigation.
- Level 3+ autonomy features requiring no human supervision or full autonomy.
- Operation in adverse weather conditions (rain, fog, nighttime).
- High-definition mapping and global route planning.
- Highway or high-speed driving scenarios and uncontrolled public traffic environments.

- Operation in environments without visible lane markings
- Vehicle-to-Vehicle (V2V) or Vehicle-to-Infrastructure (V2I) communication.
- Cybersecurity hardening for production deployment
- Autonomous parking systems.

Chapter 2

Literature Review

2.1 Previous Work

2.1.1 Autonomous Driving Levels and L2 Systems

Classification and Evolution of Autonomy

The SAE J3016 standard serves as the foundational framework for the automotive industry by defining six distinct levels of driving automation. It provides a technical roadmap that tracks the transition from manual driving to full machine-led autonomy while establishing clear boundaries between human and system responsibilities.

- **Hierarchical Taxonomy:** The system is organised into levels 0 through 5, where the role of the human driver progressively diminishes as the automation level increases.
- **Functional Distinction:** Levels 0, 1, and 2 fall under Driver Support Features, which always require active monitoring and control by the human. Levels 3, 4, and 5 fall under Automated Driving Features.
- **The Level 2 Integration:** A major milestone was the shift from Level 1 (single-task assistance) to Level 2 (multi-task assistance), where systems simultaneously manage both steering and speed control.
- **Evolutionary Drivers:** The progression from early ADAS to high-level autonomy has been powered by the integration of Convolutional Neural Network (CNN), LiDAR, and sensor fusion to handle complex urban navigation. [1].

Tesla Autopilot

Tesla stands out in the ADAS market by relying strictly on Computer Vision rather than multi-sensor fusion. While this approach mimics human sight, it carries a unique set of advantages and risks.

- **AI Models:** Tesla uses Self-Supervised Learning and End-to-End Neural Networks to simplify decision-making.

- **Fleet Intelligence:** Millions of vehicles act as data probes, feeding edge cases back to Tesla to improve the global model.
- **Dynamic Planning:** Advanced algorithms predict the movement of other road users to navigate complex highway and city interchanges.

Challenges and Limitations

Tesla's system is officially categorised as SAE Level 2, meaning the driver is legally responsible for the vehicle at all times.

- Tesla's camera-only solution performs poorly in adverse weather conditions, such as fog, rain and night time.
- Unexpected breaking and navigation errors in urban areas have prompted regulatory scrutiny and recalls. [6, 7].

GM Super Cruise

GM Super Cruise is an SAE Level 2 autonomous system that enables hands-free driving through a combination of precision LiDAR mapped data, camera-based lane centring, and radar-based Adaptive Cruise Control (ACC). GM's Super Cruise stands as a prominent example of a "hands-off, eyes-on" Level 2 system, utilizing robust driver monitoring and pre-mapped data to enhance safety on highways.

- **LiDAR-Mapped Road Integration:** Works on compliant Global Positioning System (GPS)-mapped roads like limited-access freeway and trunk roads to ensure high-fidelity navigation.
- **Sensor Fusion Architecture:** Employs a combination of Radio Detection and Ranging (RADAR) and camera sensors for ACC to manage vehicle speed and following distances.
- **Safety Performance:** Real-world Telematics data indicates that hard braking (above 2.6 m/s^2) is 1.7 times more frequent during manual driving than when Super Cruise is engaged.

Challenges and Limitations

- The system is limited to specific mapped highways; 58% of takeover requests are triggered by the vehicle reaching these geographic or system limits. [8].

Ford BlueCruise

Ford's BlueCruise is an SAE Level 2 active driving assistance system that enables hands-free operation through the simultaneous integration of ACC and Lane Centring Assistance (LCA).

- **Hands-Free Blue Zones:** Allows for hands-off-wheel operation of vehicles upon more than 130,000 miles of pre-mapped highways across North America.

- **Collaborative Steering:** BlueCruise is one of the only systems that will let the driver manually steer the vehicle without automatically turning off the lane centring feature.
- **Predictive Speed Control:** Controls via ACC to keep an appropriate distance from the traffic flow in front of the vehicle and to deceleration when approaching slower traffic

Challenges and Limitations

- Although it works on non-highway roads, it has to switch from 'hands-free' to 'hands-on' driving assistance in regions out of Blue Zones. [9].

2.1.2 System Architecture and Sensor Fusion Architectures

Autonomous driving systems adopt layered and modular software architectures to manage perception, decision-making, and control complexity. Sensor fusion strategies are commonly categorized as early, late, or hybrid fusion. Waymo employs early fusion by integrating raw LiDAR point clouds with synchronized camera data, improving spatial reasoning and object localization accuracy [9]. In contrast, Tesla's Full Self-Driving system follows a vision-centric late fusion paradigm, where camera streams are processed independently by deep neural networks before integration at the decision level [10]. Hybrid fusion architectures balance robustness and efficiency by selectively combining complementary sensor modalities [11].

For SAE-Level 2 systems, modular design architecture is more relevant, as it facilitates autonomy development, testing, and module isolation in perception, planning, and control functions. ROS 2, has proven to be the preeminent solution among vehicle research communities dealing with autonomous vehicles and offers distributed processing, deterministic communication, message interfaces, and time synchronization that are essential in multi-sensor integration [12].

Current studies suggest the importance of learning-based fusion methods that perform fusion operations at a higher semantic level using information from a multitude of sensors. However, such fusion methods face high computational complexity requirements, thereby proposing the necessity to co-design the hardware.

2.1.3 Sensor Technologies

Modern ADAS utilize a set of diverse sensors in order to provide a robust perception environment as depicted in Figure 2.1. By fusing complementary sensing modalities that compensate for the limitations of individual sensors, improving reliability in varying light, weather, and traffic settings. Camera-based systems continue to occupy a prominent position and vary in complexity, such as in Tesla Autopilot and multi-camera systems in Waymo, which provide nearly 360° coverage [10, 11]. Monocular systems are relatively inexpensive and efficient in computation but not able to provide depth information and are susceptible to variations in illumination. The stereo and multi-camera approaches can overcome these problems through geometric reasoning but are more computationally expensive and challenging in terms of calibration procedures [13].

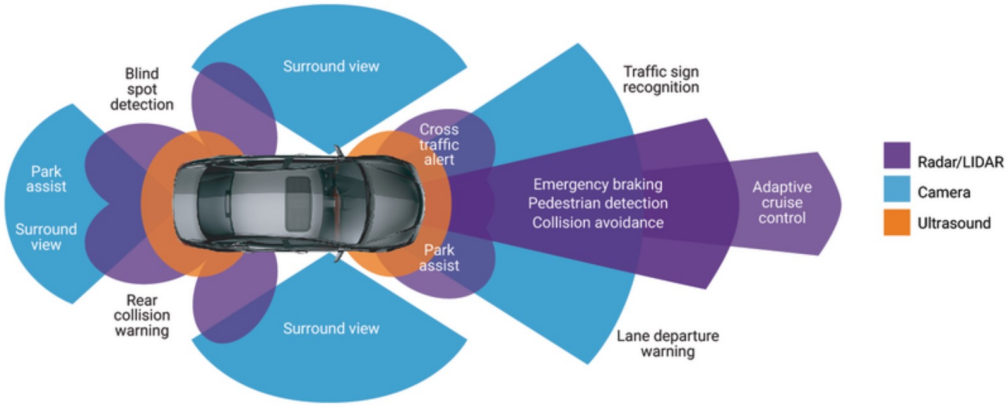

Figure 2.1: Multi-sensor coverage and functional mapping in a Level-2 ADAS.

LiDAR sensors are complementary to vision sensors as these sensors enable precise three-dimensional geometric measurements irrespective of the environment lighting conditions. Though initial autonomous vehicles use LiDAR systems with mechanically rotating LiDAR devices, nowadays solid-state LiDAR configuration systems are dominating due to reliability and automotive adaptability issues [14]. The millimeter wave radar sensor further adds reliability as it offers precise range and relative velocity measurement irrespective of deterioration due to harsh environmental conditions that degrade optical sensors and are essential for SAE Level-2 ACC functions [13].

Accurate spatial and temporal calibration is crucial for effective multi-sensor perception. Hardware-synchronized acquisition combined with precise timestamping allows for proper fusion, while in-vehicle networks like CAN support real-time control with latency low enough for most safety functions [12].

2.1.4 Perception and Computer Vision

Vision-based perception provides a basis for modern ADAS technology, supporting detection, recognition, tracking, and free-space analysis. Lane detection began from classical image processing methods. However, the recent trends in image processing apply Deep Learning methods to image analysis tasks, including image detection. Convolutional neural networks have been proven to offer high resistance to normal working conditions [13]. Semantic segmentation methods offer a pixel-level analysis of the environment, thereby supporting accurate detection for drivable regions.

Object detection in automotive applications favors systems which offer a better compromise between real-time performance and accuracy. Single-stage detectors gain popularity as they provide a more desirable trade-off between latency and error rate compared to their safety-critical counterparts, which usually employ redundancy to enhance robustness [13]. The perception software stack provided by Tesla is a great example of a multi-task learning framework, which handles multiple tasks like object detection, segmentation, and depth prediction in a single network to enhance computational efficiency [11].

Multi-object tracking fuses perception outputs with motion models, provid-

ing essential temporal consistency for trajectory forecasting and collision avoidance. Most production and research methods rely on the tracking-by-detection paradigm, where learned appearance features are combined with probabilistic motion models [13]. While great progress has been made, perception systems are still sensitive to dataset bias and domain shifts when the models of interest, so to say-trained in simulation or limited geographical areas-are tested in the real world, which requires continuous validation and sim-to-real adaptation.

2.1.5 Path Planning and Decision Making

Path planning and decision making form the core cognitive functions of autonomous vehicles, enabling them to navigate from origin to destination while ensuring safety, comfort, and efficiency. These functions are typically structured in a hierarchical framework comprising four key layers:

- **Global Planning:** Determines overall route from start to destination based on road networks, traffic, and user preferences. Generates a coarse path without considering dynamic obstacles or real-time traffic.
- **Behavior Planning:** Tactical layer making high-level decisions about lane changes, overtaking, merging, and intersections.
- **Local Path Planning:** Operational level, generating collision-free trajectories within the immediate environment. Accounts for dynamic obstacles, road boundaries, and kinematic constraints.
- **Motion Prediction:** Forecasts trajectories of surrounding dynamic objects, enabling proactive planning and safer decision-making.

Automation Framework and Scope: The SAE J3016 standard defines where strategic actions such as destination decision-making, trip planning, and waypoint decision-making are outside the scope defined by automation levels 0-5 and instead focuses on driving automation at a tactical and operational level defined by planning, local path planning, and motion prediction[1]. Modern research endeavors on autonomous vehicles cover these operation defined levels using hierarchical architectures which encompass decision-making algorithms and real-time sensor processing abilities.

Behavior Planning and Decision Making: FSM architectures demonstrated effectiveness in multi-agent avoidance scenarios; Naranjo et al. achieved 50Hz real-time performance on Jetson-equivalent hardware [15]. Decision tree methodologies provide structured behavior selection; Xu et al. integrated LiDAR-camera fusion for motion prediction [16].

Path Planning Methodologies: Spline-based approaches have recently become quite popular for human-aware navigation in mixed traffic. Schörner et al. [17] integrated the principles of Social Value Orientation into human-aware trajectory generation by using camera-LiDAR fusion that enables dynamic obstacle handling at 20Hz on Jetson Nano platforms. The employment of quintic splines

in their work guarantees smoothness in trajectories and real-time feasibility while accounting for social interactions.

The advantage of optimization-based methods is that they allow adaptability in dynamic environments. Reitmann et al. [18] used Timed Elastic Bands after coarse A* planning, allowing fusion of single-camera and 3D LiDAR for obstacle prediction. The real-time performance of the proposed approach surpasses 30Hz on Jetson Orin Nano hardware; it allows path density adjustment, hence scalable to embedded systems.

Motion Prediction and Trajectory Optimization Motion prediction systems utilize Kalman-filtered sensor fusion of camera point cloud data with LiDAR data. Yeong et al. [19] have shown that trajectory optimization using polynomial curves and reinforcement learning hybrids for decision-making in behavioral modeling can process sensor fusion in real time in under 50ms per layer on embedded GPU systems such as Jetson. The incorporation of reinforcement learning allows for adaptive decision-making-learning from experience.

More recently, Reachable Set Model Predictive Path Integral control methods were proposed for trajectory optimization. Liu et al. [20] showed that by utilizing LiDAR and camera occupancy grid predictions, collision-free polynomial splines can be obtained at real-time rates of 25 Hz for Orin Nano-scale hardware. This work has been identified as state-of-the-art in computationally efficient path planning.

2.1.6 Control Systems

MPC/PID controllers for longitudinal and lateral control

In the development of L2 autonomous systems, the "Adaptive Cruise Control" layer is responsible for translating perception data into physical vehicle actions.

This is split into **longitudinal control** (acceleration and braking) and **lateral control** (steering).

- **Tesla** is moving their Full Self-Driving stack towards a Model Predictive Control solution.
- **GM employs MPC** in their Adaptive Cruise Control system to accomplish "eco-driving" objectives. Moreover, GM adopts robust control architecture combining map-based feed-forward control and MPC to provide high-fidelity lane centring control.
- **Ford's BlueCruise** has traditionally utilized a very sophisticated PID (Proportional Integral and Derivative) feedback control loop with feed-forward compensation for its Intelligent Adaptive Cruise Control.

PID is a reactive controller because it continually checks the error between current and desired states and takes steps to correct it only after it has happened. It is useful in Level 2 systems because it still involves human observation as a major component. On the other hand, MPC is optimal because it looks ahead into the future based on a time horizon and, in effect, foresees what is ahead on

the road. Its ability to see ahead into potential road obstacles such as curves, traffic, and other objects makes it crucial to the decision-making process needed by advanced autopilot systems. [6–9]

Lane Keeping Assistant and Lane Centring Control

In the hierarchy of driving automation, lateral control is divided into two primary categories: reactive systems that prevent lane departure and proactive systems that maintain a continuous path within the lane.

- **Lane Keeping Assist:** This is a reactive safety feature that takes effect only when the car is about to cross lane markings without the turn signal on; it is an SAE Level 0-1 feature. It issues a rapid steering correction to push the car back into the lane.
- **Lane Centring Control:** Unlike LKA, this system continuously keeps the car centred in the lane through constant steering adjustments of minute nature. It is a key ingredient in Level 2 autonomous systems from Ford BlueCruise to GM Super Cruise, where it works in conjunction with Adaptive Cruise Control to maintain lane position and speed.
- **PID Control:** Simple, reliable, and easy to implement, with limited computing power. However, it is generally only reactive; it corrects the error after the error has occurred rather than planning ahead of it.
- **Model Predictive Control:** Now becoming the standard approach for advanced self-driving due to its capability of "looking ahead" and planning a trajectory in an optimal way regarding all upcoming curves, obstacles, and other constraints. It results in smoother driving, but requiring more computational power than PID. [8, 9, 21]

2.1.7 Level 2 Autonomous Systems Integration in Electric Vehicles

The incorporation of SAE Level 2 autonomous systems in electric vehicles' platforms comes with its own set of technical considerations. The electric vehicle platform properties, such as torque response, regenerative braking, and their battery capacity, significantly affect the design of an autonomous system, demanding different strategies for control incorporation, energy management, and CPU allocation.

Regenerative Braking Integration with Autonomous Control

The integration of functionalities such as ACC and emergency brakes according to SAE Level 2 in electric cars needs smooth interfacing with regenerative brakes. The autonomous system needs to be able to manage deceleration actions along with the motor's capability for energy regeneration for maximizing deceleration and ensuring comfort and safety simultaneously. AI-based algorithms contribute to a braking time decrease of 11.5% and also improve state-of-charge value by 0.487% over traditional friction brakes [22]. The algorithmic requirement is to

be able to mix regenerative brakes and friction brakes real-time when performing emergency brakes and maximize deceleration at the cost of efficiency.

Energy-Efficient Path Planning and Eco-Driving Strategies

SAE Level 2 systems operating in electric vehicles can apply energy-conscious control strategies, thus improving the vehicle's range. Research has indicated that optimal energy prediction yields 6.3% improvements in error margins for energy consumption estimations [23]. When considering highway operating conditions in which Level 2 systems perform well, control strategies in Model Predictive Cruise Control utilize predictions regarding grade changes and traffic conditions in order to reduce energy consumption while satisfying time constraints specified by the driver.

Battery Management Considerations for Compute-Intensive AI Workloads

The use of SAE Level 2 perception and planning software in electric cars introduces competition for resource allocation in utilizing the battery power in electric cars. The edge computing modules involved in processing camera-based object detection and processing of LiDAR and other sensors consume 200-500W of power constantly [24]. The resource-intensive processing power results in mileage losses of up to 5-10%, depending on the operation being performed. Battery thermal management becomes critical as simultaneous fast charging and compute-intensive autonomous operations can accelerate cell degradation. Intelligent power management strategies dynamically adjust algorithm complexity based on battery state, thermal conditions, and scenario criticality.

Vehicle Dynamics Modeling for Electric Powertrains

Lane centering and trajectory tracking at Level 2 on electric vehicles need adapted control models that account for the electric powertrain dynamics. Electric motors provide instantaneous torque without lag, enabling precise lateral control but requiring different stability management compared to internal combustion vehicles. The placement of the batteries changes the weight distribution and, consequently, the center of gravity, altering the characteristics of the handling that the autonomous controllers must accommodate. Energy-efficient model predictive control approaches optimize motor torque delivery across driving phases, demonstrating 11.74% improvement in energy recovery while maintaining Level 2 performance requirements [25].

2.1.8 Safety and Validation

SAE Level 2 autonomous vehicles development implies the incorporation of safety standards and validation methodologies, together with practical road testing protocols. This review examines functional safety frameworks, simulation-based testing, and critical safety mechanisms for reliable implementation using embedded platforms like NVIDIA Jetson with camera and LiDAR sensors.

Functional Safety Standards

For the safety of autonomous vehicles, two different ISO guidelines work in conjunction for safety. ISO 26262 provides a safety life cycle in the form of ASIL requirements for electrical/electronic systems in automobiles. ISO 21448, or SOTIF, deals with the safety risks that result in situations where the system does not fail but ignores situations such as an icy road when operating systems that rely on sensors [26]. Together, these standards ensure comprehensive safety coverage against malfunctions and performance limitations for camera-LiDAR configurations.

Simulation-Based Validation

Validation on a virtual simulation platform precedes road testing. Engineers subject embedded system chips to diverse scenarios, such as uncommon and critical situations, with platforms like CARLA, LGSVL, and PreScan. Test conditions are determined by operational design domain (ODD), object detectability, and failure modes. In AEB system validation, Euro NCAP protocols require TTC measures with critical limits set to TTC < 2.5 seconds for emergency action [27].

Operational Design Domain and Safety Validation

ODD identifies conditions under which the autonomous system can be operated without risk of harm, such as scenarios on highways under typical weather conditions but with challenges under adverse scenarios. SOTIF analysis of validations uses "combinatorial methods and Monte Carlo sampling" for analyzing failure rates of less than 10^{-9} in one hour to satisfy ODD boundaries [28]. For camera and LiDAR systems, it involves validating performance under conditions of sensor occlusion, weather conditions, lighting conditions, and edge cases on operation boundaries.

Road Testing Safety Mechanisms

During on-road testing of SAE Level 2 systems, test personnel and road users are protected by several layers of safety. First, there is a licensed safety driver who must be "continuously attentive to the traveled way, completely alert, and ready to take control of the vehicle with latency between 4-10 seconds when the driver is alerted to system limitations". Emergency stop functions include easy-to-reach kill switches that shut off autonomous operation in an instant. Vehicles are also required to capture and store sensor data up to at least 30 seconds before the moment of any collision [29].

Safety Architecture and Runtime Monitoring

Jetson-based systems are required to have redundant sensor fusion through Kalman filtering to maintain functionality even if sensors fail, and fail-operational planning modules are used to provide safe trajectories in systems that have partial failures. Edge computing is used, which involves processing data on vehicles, reducing the processing time, and is even up to 2 GB/s [24].

SAE Level 2 requires eye-tracking and hand-wheel observation to ensure the preparedness of the system to take control. A clean human-machine interface helps in easy communication with proper system status and escalating warning cascades to avoid over-reliance on automated control.

There is hardware-based monitoring independent of the software program state through watchdog timer implementations, which issue resets upon the loss of periodic refresh pulses. For the NVIDIA Jetson-series of platforms, nested watchdog techniques integrating timer support in the Linux kernel and at the level of the bootloader offer fail-safe capability, which is able to detect stalled algorithms in hundreds of milliseconds, while the watchdogs in the firmware prevent kernel panics by automatically switching the boot slot. This design is able to support the requirements of independent monitoring of safety-critical tasks as specified by the ISO 26262 standard.

2.2 Gaps in Literature

While the existing literature provides substantial foundations for autonomous vehicle development, several gaps remain relevant to the present project:

- **Limited Sim-to-Real Validation in Urban Scenarios:** Most published sim-to-real transfer research focuses on highway or simple track environments. Comprehensive validation of perception and control transfer in complex urban scenarios with pedestrians, traffic signals, and intersections remains limited, particularly for SAE Level 2 systems operating at low speeds.

- **Resource-Constrained Multi-Sensor Fusion:** While sensor fusion techniques are well-documented for high-performance computing platforms, optimization strategies for real-time multi-modal fusion on embedded platforms like Jetson Orin remain under-explored, particularly when integrating camera and LiDAR data for simultaneous object detection, lane keeping, and traffic signal recognition.

- **Adaptive Control Tuning for Retrofitted Platforms:** Most autonomous control research assumes purpose-built platforms with known dynamics. Methods for automatic identification and adaptation of control parameters for retrofitted electric vehicles with unknown or variable dynamics represent an important gap, as manual tuning is time-consuming and requires specialized expertise.

- **Integration of Multiple SAE Level 2 Features:** While individual components of Level 2 systems (ACC, Lane Keeping Assistant (LKA)) are extensively studied independently, research on their integrated operation, conflict resolution between subsystems, and seamless transitions between different automation modes is less comprehensive. The present project's holistic approach to implementing multiple Level 2 features simultaneously addresses this gap.

- **Low-Speed Urban Autonomy:** The majority of autonomous driving research targets highway scenarios or higher-speed operation. The specific

challenges of low-speed urban autonomy (10–30 km/h), including frequent stop-start cycles, close-proximity obstacle avoidance, and interaction with vulnerable road users, receive less attention but are critical for practical urban deployment.

These gaps inform the research approach and expected contributions of the present project, particularly in demonstrating complete system integration from simulation through real-world deployment on a resource-constrained embedded platform operating in urban scenarios.

Chapter 3

Methodology

3.1 Research Design

The autonomous vehicle system architecture follows a hierarchical decomposition consisting of four primary layers.

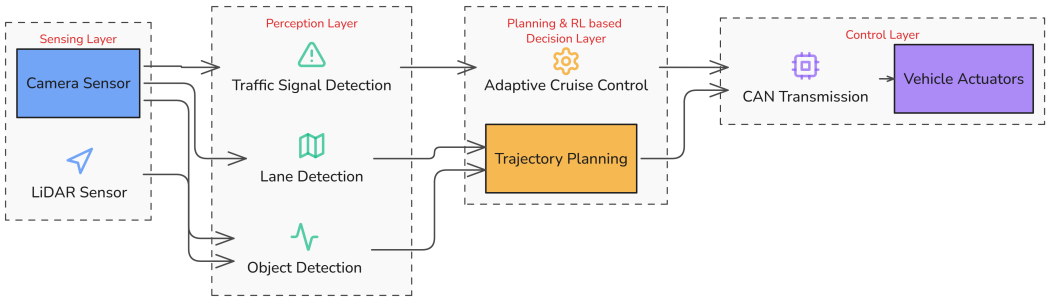

Figure 3.1: Hierarchical autonomous vehicle system architecture showing the four primary layers: Sensing, Perception, Planning, and Control.

As shown in Figure 3.1, the system consists of four distinct layers that process data from sensors to vehicle actuator commands.

3.1.1 System Architecture

Sensing Layer Physical sensors including a camera and LiDAR acquire raw environmental data. This layer includes sensor drivers, calibration data, and low-level data preprocessing.

Perception Layer Processes sensor data to extract semantic information about the environment. Modules include object detection, lane detection, and traffic signal recognition [18]. Outputs are represented as structured data describing detected entities, their properties, and spatial relationships.

Planning Layer Consumes perception outputs to generate vehicle behavior plans. Modules include trajectory planning, decision-making, and behavioral arbitration [3**?**]. Outputs consist of target trajectories expressed as sequences of waypoints with associated target speeds.

Control Layer Translates planned trajectories into vehicle actuator commands. Modules include lateral control (steering) and longitudinal control (throttle/brake) [5]. The layer interfaces with vehicle drive-by-wire systems through standardized control messages.

A central data management component maintains system state, logs sensor data and algorithm outputs for offline analysis, and coordinates timing across modules to ensure synchronized operation.

3.1.2 Simulation Environment Development

Sensor Configuration Virtual sensors in CARLA are configured to match the specifications of physical sensors used in hardware deployment. Camera parameters, including resolution, field of view, exposure, and mounting position, are matched to the Arducam specifications. LiDAR configuration, including scan rate, vertical and horizontal resolution, range, and point density, is configured to represent the physical LiDAR sensor.

Adaptive Cruise Control Adaptive Cruise Control is implemented by configuring longitudinal control logic in CARLA to replicate real-vehicle behavior, where radar-/LiDAR-based lead vehicle detection, time headway thresholds, and acceleration–deceleration limits are matched to the constraints of the physical electric vehicle platform. Vehicle dynamics parameters, including maximum braking force, throttle response, and control update rate, are tuned to reflect the real drivetrain and actuator characteristics to ensure consistent speed regulation during simulation-to-hardware transfer.

Trajectory Planning and RL-Based Decision Making Trajectory planning and RL-based decision-making modules are trained in the CARLA environment using vehicle kinematic constraints, lane geometry, and traffic interaction rules that closely approximate real-world operating conditions. The state space, action space, and reward function are designed to align with physical vehicle limits, ensuring that learned policies generate safe, feasible steering and speed commands when deployed on the real electric vehicle under supervised Level 2 autonomy.

3.2 LiDAR-Perception Module Development and Experimentation

This section explains LiDAR as a measurement technology and how it is connected to CARLA simulation software systems, LiDAR point clouds are processed to identify clusters corresponding to potential obstacles. [4]

How LiDAR Measures Distance : LiDAR is an active ranging technology. A laser emitter sends out a short, collimated pulse of light. The pulse travels outward, strikes a surface, and a fraction of the energy is reflected back to a co-located photodetector. Because the speed of light c is constant and known, the distance d to the reflecting surface follows directly from the round-trip *time of flight* (ToF) Δt:

$$d = \frac{c \cdot \Delta t}{2} \tag{3.1}$$

The factor of two is used because the pulse travels from the sensor to the target and then returns back to the sensor. Therefore, one emitted pulse provides one distance measurement along a single direction.

To create a useful representation of the surrounding environment, the emitted beam is scanned across different directions. A mechanical or solid-state system rotates or steers the beam over a range of horizontal angles (azimuth), while multiple laser channels cover different vertical angles (elevation). Each emitted pulse generates one measurement point. By combining the measured distance with the known azimuth θ and elevation ϕ angles of the beam, the polar coordinate measurement can be converted into a Cartesian coordinate system.

$$x = d\cos\phi\cos\theta, \qquad y = d\cos\phi\sin\theta, \qquad z = d\sin\phi \tag{3.2}$$

The accumulated set of (x, y, z) coordinates collected during one full sweep is called a *point cloud*. Many sensors also record the returned pulse *intensity* (reflectivity) as a fourth value per point. The number of full sweeps performed per second is the *rotation* or *scan frequency*, and the number of pulses fired per second sets the point-cloud density.

3.2.1 Research Design and Core Theory

The experiment uses LiDAR for *obstacle detection* alongside the existing monocular camera. CARLA's ray-cast LiDAR already gives each point as a ready-made (x, y, z) coordinate, so the project's code never has to do the raw travel-time-to-distance step of Equation 3.1, the simulator handles it. The project's own distance work starts after that: it groups the points into objects and measures the distance to each object. Two methods are used:

- **Euclidean clustering for obstacle detection :** Raw points are grouped into discrete obstacles using density-based spatial clustering (DBSCAN), implemented in `lidar_obstacle_detector.py`. Each cluster is reduced to a centroid, and the horizontal distance to that centroid is taken as the obstacle distance:

  $$d_{\text{obs}} = \sqrt{c_x^2 + c_y^2}, \qquad \theta_{\text{obs}} = \text{atan2}(c_y, c_x) \tag{3.3}$$

  where (c_x, c_y) is the mean of the cluster's points in the sensor's horizontal plane.

- **Projective cross-sensor association :** LiDAR distance is tied to a camera detection by projecting the LiDAR points onto the camera image. The method has two fixed ingredients: a pinhole camera model derived from the 90° FOV, and a fixed extrinsic transform. The known 0.4 m height gap and $-15°$ pitch between the two sensors. Both sensors are rigidly mounted at known positions, so this transform is hard-coded rather than estimated. Each LiDAR point is mapped to a pixel. Points landing inside a camera bounding box give that detection its LiDAR distance (their median forward depth). This is a projection not triangulation or SLAM. No mapping or

localization sub-system is built. Depth is already supplied directly by the LiDAR, and the sensor poses are already known.

Objective of the LiDAR Integration

1. **Add a geometric distance source.** Provide an obstacle-distance estimate that does not depend on the monocular camera's object-height assumption. The camera detector estimates distance from bounding-box height and a hard-coded real-world object height, which is inherently approximate. LiDAR supplies a direct measurement.

2. **Enable a quantitative sensor comparison.** A metrics logger records camera distance, LiDAR distance, and CARLA ground truth per frame and per obstacle so the two sensors can be evaluated head-to-head.

Rationale for Hardware, Tools, and Algorithms

- **Sensor model :** CARLA's `sensor.lidar.ray_cast` blueprint is used. This is a simulated ray-cast LiDAR. It runs entirely in the CARLA simulator.

- **Sensor configuration :** The blueprint attributes set in the system are listed in Table 3.1. They were chosen to emulate a mid-range automotive scanning LiDAR: a 32-channel vertical stack, a 50 m range that matches the obstacle-detection working distance, and a 20 Hz rotation rate that comfortably exceeds the control-loop cadence.

- **DBSCAN clustering :** DBSCAN was chosen because it groups points by spatial density without needing the number of clusters in advance, and labels sparse returns as noise. This is ideal for LiDAR data, which often has variable point density and may include outliers. The algorithm's parameters (epsilon and minimum samples) are tuned to balance sensitivity to small obstacles and robustness against noise in the point cloud.

- **NumPy :** All point-cloud manipulation uses NumPy arrays for vectorized filtering of large $N \times 4$ buffers.

- **OpenCV :** is used to render the bird's-eye-view (BEV) debugging canvas and the on-screen metrics panel.

- **Pinhole projection for fusion :** A simple pinhole camera model with a pitch-corrected extrinsic transform was chosen over a full calibrated projection matrix because both sensors are rigidly mounted at known transforms in simulation, so the extrinsics are exactly known.

Table 3.1: LiDAR sensor configuration, from the system implementation.

Attribute	Value	Meaning
`channels`	64	Vertical laser channels
`points_per_second`	1000 000	Pulse rate
`rotation_frequency`	20	Full sweeps per second (Hz)
`range`	100	Maximum range (m)
`upper_fov`	5	Topmost channel elevation (deg)
`lower_fov`	−25	Bottommost channel elevation (deg)
Mount transform	$x = 2.0,\ z = 1.8$	Sensor offset on ego (m)

3.2.2 Implementation of the LiDAR Perception Module

Architecture of LiDAR Integration

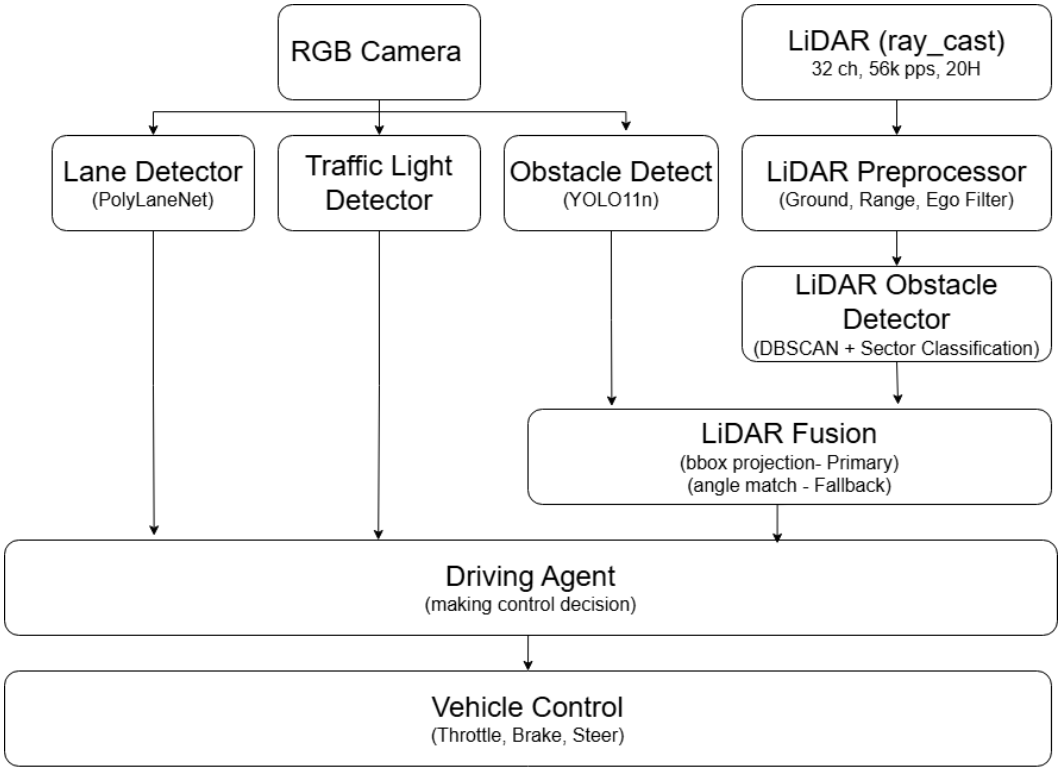

Figure 3.2: LiDAR perception module and integration architecture.

The LiDAR feature is wired into the existing `DrivingAgent`. The camera detection modules were left functionally intact; their *output* is now consumed by the new fusion stage rather than driving control directly. Figure 3.2 shows the module-level architecture.

Step-by-Step Implementation

Initialization and Configuration : The LiDAR sensor is created and configured with the attributes of Table 3.1, and wrapped in the project's `LidarSensor` class.

Reading and Parsing Raw LiDAR Data : CARLA delivers each scan as a flat byte buffer. Then buffer reinterprets the buffer as 32-bit floats and reshapes it into an $N \times 4$ matrix with columns $[x, y, z, \text{intensity}]$, in the sensor-local frame ($+X$ forward, $+Y$ right, $+Z$ up).

Preprocessing (Filtering the Point Cloud) : Raw LiDAR data is converted into a clean $M \times 3$ XYZ array by applying three raw $N \times 4$ cloud into a clean $M \times 3$ XYZ array by applying three filters in sequence:

1. **Ground removal** - discard points below `ground_z_threshold` (-1.4 m in the sensor frame, since the sensor sits 1.8 m above the road).
2. **Horizontal range filter** - keep points whose horizontal distance lies between `min_range` (0.5 m) and `max_range` (100 m).
3. **Ego-body filter** - discard self-returns within `ego_radius` (1.5 m) of the sensor.

Clustering, Distance, and Danger Classification : DBSCAN algorithm is applied (`eps` $= 0.4$, `min_samples` $= 10$) on the *horizontal* (x, y) projection of the filtered cloud. Each non-noise cluster is reduced to a centroid; the obstacle distance and bearing follow from Equation 3.3. The bearing is used to assign a *sector*: `front` ($|\theta| \leq 30°$), `side_left` / `side_right` ($30° < |\theta| \leq 150°$), or rear (discarded).

Each obstacle is classified into a danger level (`drive`, `cautious`, `slow`, `stop`, `emergency_stop`) against speed-adaptive distance thresholds. The base thresholds are 3 m (emergency), 5 m (stop), 10 m (slow), and 15 m (cautious); each is widened by a term proportional to the ego speed (`speed_factor` $= 0.5$), so faster driving triggers earlier responses. Side-sector obstacles are capped at `cautious` and never force a stop.

LiDAR-to-Camera Projection : LiDAR points projects into the camera image. Camera intrinsics are derived from the 90° FOV; the extrinsic transform accounts for the 0.4 m vertical offset between the LiDAR ($z = 1.8$) and the camera ($z = 1.4$) and the camera's $-15°$ pitch. The projection translates z, applies the inverse pitch rotation, and then performs a pinhole projection:

Camera–LiDAR Fusion : For each camera detection it attempts, in priority order as below:

1. **Bounding-box projection** (`FULL_BBOX`) - project raw LiDAR points into the camera frame and take the median depth of points inside the box. This is the preferred method.
2. **Angle fallback** (`FULL_ANGLE`) - if too few points project into the box, match the detection to the nearest front-sector LiDAR cluster within a 15° horizontal gate, additionally requiring that the two distances agree to within 40 %.

3. **Camera-Only** (`CAMERA_ONLY`) - no LiDAR match; the monocular distance is retained. This data will be logged as labeled as **Camera-Only** but not used for the experiment.

Front-sector LiDAR clusters that match no camera detection are still emitted as synthetic `LIDAR_ONLY` obstacles, so a LiDAR-only objects also will be logged as well. This allows for a comprehensive comparison of the two sensors' detection capabilities.

3.2.3 Lane Keeping Assistance Development

The Lane Keeping Assistance system maintains vehicle position within the detected ego-lane through continuous steering adjustments [30]. A cascaded control architecture is employed with separate controllers for lateral error and heading error.

Lateral Controller A PID controller minimizes lateral offset from the lane center. The controller input is the lateral distance (in meters) between the vehicle center and the detected lane center at a look-ahead distance (typically 10–15 meters). The controller output is a target steering angle.

Heading Controller A secondary PID controller minimizes heading error (angle between vehicle heading and lane direction). This controller improves tracking through curves and reduces oscillation around the lane center.

Control Fusion The final steering command combines outputs from lateral and heading controllers through weighted summation. Weights are tuned to achieve a critically damped response with minimal overshoot.

Speed Adaptation Controller gains are adapted based on vehicle speed, as steering response characteristics change with velocity. At lower speeds, higher gains provide more responsive correction, while at higher speeds, reduced gains prevent excessive control action.

Safety Limits Steering commands are limited to maximum rates and magnitudes compatible with vehicle kinematics and passenger comfort. If lane detection confidence drops below a threshold, the controller reduces gain or disengages entirely, alerting the driver to resume manual control.

3.2.4 Trajectory Planning and Decision-Making Development

Planning and Decision-Making

This module is charged with the responsibility of translating perception output on how safe and smooth vehicle motion should look like under Level 2 supervision. The proposed method adopts a hierarchical architecture that distinguishes between the decision-making and trajectory planning phases and is integrated in such a way that it is safe and real-time feasible.

Decision-Making Module This module analyzes and makes decisions regarding the current driving scenario to decide upon appropriate longitudinal driving maneuvers. It runs at a higher level of abstraction.

Key responsibilities include:

- Identifying the current traffic context, such as free-flow driving, dense traffic, lead-vehicle following, or approaching traffic signals.
- Selecting the appropriate driving mode, including cruising, adaptive following, smooth deceleration, or full stop.
- Assessing risk levels associated with detected objects using perception outputs, including object class, distance, and relative velocity.
- Adapting safety margins such as time-gap and braking aggressiveness based on traffic density and perceived risk.
- Incorporating short-horizon motion prediction of surrounding vehicles to anticipate potential conflicts.

Decision-making is implemented using a hybrid approach:

- Rule-based logic enforces traffic rules, safety constraints, and system operational limits.
- A reinforcement learning (RL) component is used for adaptive parameter tuning, particularly for time-gap adjustment and object risk scoring, without performing direct control actions.

The states of the agent are a compact state space including the speed of the ego vehicle, relative distance and velocity of lead vehicles, traffic density indicators, and signal states. The objectives of the reward function are to maximize safety, smoothness, and efficacy, while discouraging aggressive acceleration, unsafe gap acceptance, and unnecessary stops.

3.2.5 Trajectory Planning Module

The trajectory planning module generates a feasible and smooth longitudinal trajectory based on the selected driving mode and safety parameters provided by the decision-making layer.

Key responsibilities include:

- Generating target speed profiles along the ego-lane that maintain safe following distances and comply with traffic signals.
- Ensuring smooth transitions between acceleration, cruising, and deceleration to enhance passenger comfort.
- Respecting vehicle dynamic limits, including acceleration, jerk, and braking constraints.

- Adjusting trajectory optimization cost weights based on risk levels supplied by the decision-making module, where higher perceived risk increases penalties on speed, acceleration, and proximity to obstacles, resulting in more cautious and conservative motion profiles.

Trajectory planning is formulated as a local planning problem and implemented using:

- Lane-centerline-based planning for Level 2 operation.
- Polynomial or spline-based trajectory generation to ensure continuity in velocity and acceleration.
- Model Predictive Control (MPC) for longitudinal trajectory optimization over a receding horizon, enabling real-time adjustment to dynamic traffic conditions.

Integration with Adaptive Cruise Control The result from the trajectory planning module is the target speed and acceleration to the Adaptive Cruise Control (ACC) module. This is followed by the execution of the targets from the ACC module while ensuring a safe distance from the preceding vehicles as well as traffic signals.

Hierarchical decentralization of decision-making tasks and trajectory planning reduces modularity issues, interpretability, and validation of security by incorporating elements of learnability that do not deteriorate deterministic control scenarios.

3.2.6 Adaptive Cruise Control Development

The Adaptive Cruise Control (ACC) system regulates vehicle speed to maintain safe following distances from lead vehicles while tracking a driver-specified target speed when no lead vehicle is present [1, 31].

Lead Vehicle Tracking Object detection and LiDAR data are fused to identify vehicles in the ego-lane ahead of the autonomous vehicle. A Kalman filter tracks lead vehicle position and velocity, providing filtered estimates that reduce sensor noise.

Following Distance Policy Safe following distance is computed using a time-gap policy where the target gap is proportional to current vehicle speed (typically 2–3 seconds). This approach naturally increases following distance at higher speeds while maintaining closer gaps at low speeds.

Longitudinal Control A Model Predictive Control (MPC) approach is employed for longitudinal control [31], optimizing the predicted vehicle trajectory over a receding horizon (typically 2–3 seconds). The optimization objective balances tracking target speed, maintaining a safe gap to the lead vehicle, passenger comfort (limiting acceleration and jerk), and fuel efficiency.

Deceleration for Traffic Signals When a red traffic signal is detected, the ACC system computes the required deceleration to stop at the signal stop line. Deceleration is applied smoothly beginning at a distance that allows comfortable braking rates (typically 2–3 m/s^2).

Speed Limits and Curvature The ACC system respects speed limits detected from traffic sign recognition and reduces speed for sharp curves based on estimated curve radius and assumed lateral acceleration limits [**?**].

Chapter 4

Testing and Performance Evaluation

This chapter evaluates the LiDAR integration by comparing the LiDAR-measured obstacle distance and the camera-measured obstacle distance, frame by frame, against the CARLA ground-truth distance. The running system logs every per-obstacle measurement to CSV; the logs are then filtered to remove noise and analyzed offline. The sections below describe the experimental setup, how the two sensors are made to measure *the same* obstacle, the KPIs used, and the analyzed results.

4.1 Experimental Setup

- **Simulator and map** - CARLA, town `Town04`.
- **Weather** - initial `ClearNoon`
- **Ego vehicle** - `tesla model3`, with a co-mounted RGB camera ($x = 2.0,\ z = 1.4$, pitch $-15°$) and LiDAR ($x = 2.0,\ z = 1.8$).
- **Traffic** - 50 moving vehicles and 10 static in-lane obstacles places static obstacles at 50, 80, 120 m ahead along the ego lane.
- **Controlled test target** - a `LeadVehicleController` can spawn a red lead vehicle 30 m ahead whose bumper-to-bumper distance serves as a clean ground-truth reference.
- **Logging** - `DistanceMetricsLogger` streams every per-obstacle measurement to CSV.

4.1.1 Obstacle Identity Matching Across Sensors

For the comparison to be valid, the camera distance and the LiDAR distance in any given row must refer to *the same physical obstacle*. The system guarantees this with a per-obstacle identity (`obstacle_id`) that is assigned during fusion and carried into the log.

1. **LiDAR track IDs :** Gives every clustered obstacle a persistent string ID - `t0`, `t1`, ... Across frames a new cluster is matched to an existing track by sector and angular proximity (a $\pm 10°$ window), so the same physical object keeps its `t`-ID for as long as it remains visible.

2. **Linking a camera detection to a LiDAR track :** Computes the horizontal bearing of each camera (YOLO) detection from the image centre and compares it with every LiDAR cluster's bearing. When a LiDAR track falls within the 15° angular gate, the camera detection *inherits that track's* `t`*-ID* as its `obstacle_id`. The angle-fallback path additionally requires the two distances to agree within 40 %, which prevents two different objects that merely share a bearing from being fused into one identity.

3. **Unmatched detections :** A camera detection with no LiDAR match receives a camera-side ID. An unmatched LiDAR cluster keeps its own `t`-ID. But labeled as **LiDAR-Only** and **Camera-Only** respectively, so they are filtered out of the comparison.

4. **Detection-source tag.** As each per-obstacle row is written, logger stamps it with a `detection_source`: `BOTH` when the obstacle was matched by both sensors (`match_method` `FULL_BBOX` or `FULL_ANGLE`), otherwise `CAM_ONLY` or `LIDAR_ONLY`.

A `BOTH`-tagged row is therefore the unit of comparison: a single `obstacle_id` together with the camera's independent distance estimate, the LiDAR's independent distance estimate, and the ground truth, all for the same object in the same frame.

4.1.2 KPIs and Metrics

The comparison uses three distance-accuracy metrics and four latency metrics. For error samples $e_i = |d_i - g_i|$, where d_i is the sensor distance and g_i the ground truth, with mean error $\bar{e}$:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} e_i^2} \tag{4.1}$$

$$\text{Mean Relative Error} = \frac{1}{n}\sum_{i=1}^{n} \frac{|d_i - g_i|}{g_i} \times 100\,\% \tag{4.2}$$

$$\text{Error Std. Deviation} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} (e_i - \bar{e})^2} \tag{4.3}$$

RMSE captures overall accuracy and penalizes large errors; mean relative error expresses accuracy as a percentage of the true distance; error standard deviation measures how *consistent* the errors are, a low value means a predictable bias, a high value means scattered errors that are hard to correct. The four latency

metrics (median, standard deviation, maximum, and 95th percentile of the per-frame processing time) characterize both the typical speed and the worst-case jitter of each pipeline. Table 4.1 summarizes the set.

Table 4.1: KPIs used in the camera-versus-LiDAR comparison.

KPI	**Meaning**
RMSE (m)	Root-mean-square distance error vs. ground truth.
Mean relative error (%)	Mean absolute error as a percentage of true distance.
Error std. deviation (m)	Spread of the error; consistency of the estimate.
Median latency (ms)	Typical per-frame pipeline processing time.
Latency std. deviation (ms)	Frame-to-frame jitter of the processing time.
Max latency (ms)	Worst observed per-frame processing time.
95th-percentile latency (ms)	Near-worst-case latency, for timing budgets.

Validation is purely simulation-based. Ground truth is obtained from CARLA actor geometry. A function gathers the nearest front vehicle's ground truth distance by using obstacle ID and returns the bumper-to-bumper longitudinal gap, not from a physical measuring instrument.

Table 4.2: Verification test cases in `test_obstacle_ids.py` and their significance.

Test case	**Significance**
FULL_ANGLE match	Camera detection near a LiDAR track inherits the LiDAR track ID — confirms angle-gate association.
CAMERA_ONLY (no LiDAR)	With no LiDAR obstacle, the detection gets a synthetic `cam_N` ID.
CAMERA_ONLY (outside angle)	A LiDAR cluster beyond the 15° gate is correctly *not* matched.
LIDAR_ONLY	A LiDAR cluster with no camera detection is still emitted as an obstacle.
Cross-frame LiDAR persistence	A track keeps the same ID across consecutive frames.
Camera-only persistence	A camera-only obstacle keeps its ID while in view.
Stale eviction	A camera-only ID is dropped after 5 unseen frames.
Multiple obstacles	Two obstacles at different angles get distinct, correct IDs.
FULL_BBOX match	Raw points inside a bbox yield a bbox-projected match.

Each in-simulation run is stored as a dated session in the `metrics/` directory.

The results below are computed from the filtered `BOTH`-tagged rows of such a session, following the noise-filtering procedure described above.

4.2 Results and Analysis

4.2.1 Distance Accuracy

Table 4.3 reports the three distance-accuracy metrics for the camera and the LiDAR, each measured against the CARLA ground truth on the filtered `BOTH`-tagged rows.

Table 4.3: Distance accuracy vs. CARLA ground truth (filtered `BOTH`-tagged rows).

Metric	**Camera**	**LiDAR**
RMSE (m)	3.95	3.46
Mean relative error (%)	6.14	5.20
Error std. deviation (m)	3.28	3.00

LiDAR is the more accurate distance source on every metric: its RMSE of 3.46 m is about 12 % lower than the camera's 3.95 m, and its mean relative error (5.20 %) is below the camera's (6.14 %). Both sensors, however, stay within a single-digit relative error, so the camera remains a credible backup. The more important observation is that for *both* sensors the error standard deviation (3.00 m and 3.28 m) is of the same order as, and in fact exceeds, the typical error itself. The errors are therefore not a fixed, easily-corrected bias but are scattered with a long tail of outliers, a single-frame reading from either sensor should not be trusted directly for a control decision.

4.2.2 Processing Latency

Table 4.4 reports the per-frame processing latency of the two detection pipelines, computed from the `cam_latency_ms` and `lidar_latency_ms` columns logged by the timing instrumentation in driving agent.

Table 4.4: Per-frame processing latency of the camera and LiDAR pipelines.

Metric	**Camera**	**LiDAR**
Median latency (ms)	12.78	1.72
Latency std. deviation (ms)	3.57	1.27
Max latency (ms)	21.86	7.20
95th-percentile latency (ms)	20.25	4.67

The LiDAR pipeline is dramatically faster, a median of 1.72 ms against the camera's 12.78 ms, roughly a 7× speed-up and far more predictable, with a 95th-percentile latency of 4.67 ms versus the camera's 20.25 ms. The camera pipeline shows pronounced jitter (standard deviation 3.57 ms, maximum 21.86 ms).

Chapter 5

Discussion and Insights

5.1 Summary of Key Results and Observations

- **Accuracy :** On the filtered `BOTH`-tagged rows, LiDAR is the more accurate distance source: 3.46 m RMSE versus the camera's 3.95 m (about 12 % better), and 5.20 % mean relative error versus the camera's 6.14 %.
- **Latency.** The LiDAR pipeline runs at a 1.72 ms median, roughly 7× faster than the camera's 12.78 ms and is far more predictable (95th-percentile 4.67 ms versus 20.25 ms).
- **Error consistency.** For both sensors the error standard deviation (3.00 m LiDAR, 3.28 m camera) exceeds the typical error, indicating scattered errors with a long tail of outliers rather than a simple correctable bias.

5.2 Insights and Engineering Decisions

Table 5.1 consolidates the findings of the testing chapter into the engineering decisions they motivate for a production-oriented perception stack.

Table 5.1: Insights from the camera–LiDAR comparison and the decisions they drive.

Category	**Finding**	**Implication / decision**
Accuracy (RMSE)	LiDAR 3.46 m vs. camera 3.95 m. LiDAR ∼12 % more accurate.	Use LiDAR as the primary distance sensor.
Mean relative error	Camera 6.14 % vs. LiDAR 5.20 % Both within an acceptable range for mid-distance detection.	Camera must for semantic understanding.
Error consistency (std. dev.)	Camera 3.28 m, LiDAR 3.00 m, the spread exceeds the mean error, indicating scattered errors with long-tail outliers.	Do not trust single-frame readings; apply Kalman or moving-average smoothing before control decisions.
Sensor fusion	Similar error spreads but different failure modes — the camera struggles with depth, LiDAR with low-reflectivity returns.	Fuse both sensors with inverse-variance weighting (≈52 % LiDAR / 48 % camera).
Latency median	LiDAR 1.72 ms vs. camera 12.78 ms. LiDAR is ∼7× faster.	LiDAR is suited to the real-time / emergency-braking loop.
Latency 95th pct.	Camera spikes to 20.25 ms; LiDAR stays under 4.67 ms.	The camera pipeline has jitter, investigate frame-processing bottlenecks or budget for the worst case.
Latency max	Camera peaks at 21.86 ms vs. LiDAR 7.20 ms.	Run the camera and LiDAR on separate threads so camera spikes cannot block the LiDAR loop.
Safety margin	$2\sigma \approx 6$–7 m of uncertainty at the 95 % confidence level for both sensors.	Build braking-distance buffers with ∼6 m margin; never plan trajectories at the edge of detection range.
Overall	LiDAR-primary, camera-secondary, with temporal smoothing and sensor fusion.	Production stack: LiDAR for the fast loop, a fused estimate for path planning, the camera for classification and redundancy.

5.2.1 Objectives Achieved?

1. **Add a geometric distance source — achieved.** LiDAR supplies a direct, camera-independent obstacle distance, and on cleanly matched obstacles it is clearly the more accurate of the two sensors.

2. **Make obstacle response conservative — achieved.** The fused action

is the worst case of camera and LiDAR, and the synthetic `LIDAR_ONLY` path lets LiDAR alone trigger a response. The control logic in `driving_agent.py` now reacts to this fused action.

3. **Enable a quantitative sensor comparison — achieved with caveats.** The metrics infrastructure exists and produces usable per-obstacle data, but the low `BOTH`-row count (51 of 902) means the clean comparison rests on a small sample, and the ground-truth association still produces outliers that must be filtered manually.

5.2.2 A Plausible Real-World Application

The most direct real-world transfer of this implementation is a **forward-collision-warning and automatic-emergency-braking (AEB)** function for an entry-level ADAS. A vehicle equipped with a windscreen camera and a low-cost forward LiDAR could run exactly this pipeline: the camera classifies *what* is ahead, the LiDAR measures *how far* it is, and the conservative fusion ensures braking is triggered if *either* sensor sees danger. The fast ($\sim$1 ms) LiDAR clustering path is well suited to a resource-constrained automotive ECU, and the asymmetric temporal filtering directly addresses the real nuisance-braking problem (a single noisy far reading must not cancel a brake request).

5.2.3 Limitations and Drawbacks

The following limitations are evident from the code and the logged results.

- **Single-plane reasoning.** Although the sensor is 3D (32 channels), `LidarObstacleDetect` clusters only the (x, y) projection and reasons about a 2D horizontal plane. Height information is collected (the z extents) but not used for classification.
- **Configured range and density are modest.** The 50 m range and 56 000 pts/s rate (Table 3.1) are far below production LiDAR. Distant objects return few points, which directly causes the low LiDAR detection rate (0.10–0.27).
- **Sparse-return failure of bbox projection.** Bounding-box fusion needs enough projected points inside the box; for distant, narrow, or occluded objects it falls through to the angle gate or to camera-only. This is the documented cause of the camera "safe-by-default" fallback.
- **Ground-truth association is fragile.** `_per_obstacle_gt()` matches detections to CARLA actors by angle and distance with fixed gates; when it picks the wrong actor it stamps a distant vehicle's range onto a near obstacle, producing long-tail error outliers in the raw per-obstacle log that must be filtered out before the accuracy metrics can be computed.
- **Small clean-comparison sample.** Only 51 of 902 per-obstacle rows were `BOTH`-confirmed, so the favourable LiDAR accuracy result rests on a thin slice of the data.

- **No clustering when `sklearn` is missing.** If `scikit-learn` is unavailable, `LidarObstacleDetector` disables clustering entirely and the LiDAR contributes nothing — a hard dependency in practice despite the guarded import.
- **Single most-recent frame only.** `LidarSensor` keeps only the latest sweep; if the control loop stalls, an old cloud is silently reused with no staleness check beyond the stored timestamp.
- **Environmental conditions are simulated.** All weather effects are CARLA presets. Real LiDAR degradation in rain, fog, dust, or from retro-reflective surfaces is not represented, so the accuracy figures are optimistic relative to real hardware.

5.2.4 Concluding Remark

LiDAR was integrated as a fast, geometrically grounded second opinion layered onto an existing camera perception stack. Its introduction was localized to a set of new `core/` modules but rippled outward: the driving agent's control input became a fused two-sensor verdict, new ground-truth and metrics machinery was added, and the visualization gained LiDAR-specific overlays. The measured outcome is a pipeline that is fast and, when both sensors agree, distance-accurate — but whose coverage is currently limited by a modest sensor configuration and a fragile ground-truth association step.

Chapter 6

Timeline and Resource Required

6.1 Timeline

6.1.1 7th Semester (December 2025 - April 2026)

Table 6.1: Timeline of activities for the 7th semester (December 2025 - April 2026)

Tasks	Dec				Jan				Feb				Mar				Apr			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
Grouping and Project Selection	■	■	■																	
Background Study				■	■	■														
Proposal Report and Presentation						■	■													
Get Familiar with Phase 1 setup								■	■											
Creating ROS Nodes for Models in Jetson Orin										■	■	■								
Camera Integration Testing Existing Models													■	■						
CAN Bus Integration															■	■				
Integration Testing & Analysis																	■	■		
Enhance Lane Detection & Implement Lane Keeping Assistant																			■	■
Simulation Testing & Integration Testing																			■	■

6.1.2 8th Semester (May 2026 - September 2026)

Table 6.2: Timeline of activities for the 8th semester (May 2026 - September 2026)

Tasks	May				June				July				Aug				Sep			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
Enhance Traffic Light Detection & TensorRT Optimization																				
Simulation Testing for LiDAR																				
Enhance Object Detection																				
LiDAR Integration & Testing																				
Implement the Decision Making RL Model and Trajectory Planning																				
Simulation Testing of Decision Making Algorithms																				
Implement ACC																				
Simulation Testing for ACC																				
Sim-to-Real Testing for Whole Integrated System																				
Final Documentation and Paper Submission, and Demonstration																				

6.2 Resources Required

This section details the estimated cost allocation for physical implementation of SAE level 2 ADAS. The cost estimates below serve as a baseline to ensure that the project remains within budget during both development and deployment phases.

Table 6.3: Estimated hardware expenditure for the development and validation of the proposed ADAS system

Cost Item Hardware	**Description**	**Cost (LKR)**
Arducam xISP - Swift AR0234 Camera	2.3 MP global shutter camera used for lane detection, object detection, and traffic signal recognition. Compatible with NVIDIA Jetson platforms.	Rs. 45,493.49
MCP2515 CAN Controller	SPI-based CAN controller module for interfacing Jetson with vehicle ECU and drive-by-wire system.	Rs. 400.00
Assembled EV vehicle chassis	EV platform with motor, steering, battery, and frame for SAE Level-2 ADAS integration.	Rs. 1,145,020.80
LiDAR	Used for depth estimation, obstacle detection, and sensor fusion (LiDAR + camera) in ACC and collision avoidance.	Rs. 185,725.92
Jetson Orin Nano	Edge AI computing unit for real-time perception, planning, and control using ROS 2.	Rs. 77,000.00
MicroSDXC (SD 3.0)	Bootable 64 GB microSDXC storage with UHS-I speed class used as the primary OS and data storage medium for the Jetson Orin Nano developer kit.	Rs. 6,296.00
Computing Resources	High-performance workstation for CARLA simulation and model training.	Rs. 556,868.22

Chapter 7

Conclusion

This project offers a transparent and modular approach for realizing a Level 2 automated driving assistance system catering specifically to urban scenarios. Based on an existing simulation-based Level 1 system developed in Phase I, Phase II enhances the system for supervised partial automation using reinforcement learning to enable and assist, but not replace, deterministic control decisions. The hierarchical architecture allows for easy separation and development along sensing and perception, planning and decision-making, and control functions, facilitating efficient development and integration testing. The perception layer combines camera and LiDAR inputs for reliable and semantic outputs corresponding to detection of objects, lanes, and traffic signals in the operational domain. Decision-making and planning are based on hierarchical planning utilizing rule-based safety planning and learning-based adaptation, and also utilize trajectory planning based on optimization techniques addressing safety, comfort, and optimization objectives. Reinforcement learning is constrained to adaptive parameter tuning to preserve predictability and safety. Model predictive control governs longitudinal and lateral dynamics through the adaptive cruise control module, ensuring smooth speed transitions and safe following behavior. Overall, the proposed Level 2 system offers a robust, safety-aware, and modular foundation for urban driving, with clearly defined operational limits and continuous driver supervision.

Expected Outcomes

- A functioning Level 2 autonomous system demonstrated on a real electric vehicle
- Open-source software frameworks enabling replication by other researchers
- Comprehensive datasets (simulation and real-world) for further research
- Validated methodologies for sim-to-real transfer in autonomous driving
- Publications contributing to RL deployment and safety assurance literature

This project positions our team at the forefront of autonomous vehicle innovation, combining rigorous engineering with cutting-edge AI research to advance the state-of-the-art in intelligent transportation systems.

References

[1] "Taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles," Apr. 2021.

[2] T. P. Swaminathan *et al.*, "Benchmarking deep learning models on nvidia jetson nano for real-time systems," *Procedia Computer Science*, 2024. arXiv:2406.17749.

[3] Anonymous, "Decision-making in autonomous driving using reinforcement learning." `https://research.chalmers.se/publication/526543`, 2023.

[4] S. Pliam, "A review of lidar-camera fusion for robust perception in complex environments." Medium, 2023.

[5] A. Bimbraw *et al.*, "Mechatronics and remote driving control of the drive-by-wire for a vehicle," *Sensors*, vol. 20, no. 4, p. 1216, 2020.

[6] R. Zhang and X. Ding, "A study on the impact of tesla's autonomous driving technology on its performance," *International Journal of Global Economics and Management*, vol. 5, no. 2, pp. 72–78, 2024.

[7] S. Nordhoff *et al.*, "(mis-)use of standard autopilot and full self-driving (fsd) beta," *Frontiers in Psychology*, vol. 14, 2023.

[8] D. LeBlanc *et al.*, "Field study of the level 2 super cruise using telematics data," in *Proceedings of the International Technical Conference on Enhanced Safety Vehicles (ESV)*, 2023.

[9] M. Monticello, "Ford's bluecruise remains cr's top-rated active driving assistance system." Consumer Reports, 2023.

[10] D. Anguelov *et al.*, "Scalability in perception for autonomous driving: Waymo open dataset," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.

[11] A. Karpathy, "Tesla autopilot and full self-driving at scale," in *CVPR Workshops*, 2021.

[12] S. Macenski *et al.*, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022.

[13] J. Janai *et al.*, "Computer vision for autonomous vehicles: Problems, datasets and state of the art," *Foundations and Trends in Computer Graphics and Vision*, vol. 12, no. 1–3, pp. 1–308, 2020.

[14] Y. Li and J. Ibanez-Guzman, "Lidar for autonomous driving: Principles, challenges, and trends," *IEEE Signal Processing Magazine*, vol. 37, no. 4, pp. 50–61, 2020.

[15] W. Xu *et al.*, "A real-time motion planner with trajectory optimization for autonomous vehicles," in *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 2061–2067, 2012.

[16] P. Schörner *et al.*, "Social value orientation-aware motion planning for autonomous vehicles," *IEEE Robotics and Automation Letters*, vol. 9, no. 10, pp. 8537–8544, 2024.

[17] S. Reitmann *et al.*, "Local path planning for autonomous vehicles based on a* algorithm and timed elastic bands." arXiv:2406.02916, 2024.

[18] D. J. Yeong *et al.*, "Sensor and sensor fusion technology in autonomous vehicles: A review," *Robotics and Autonomous Systems*, vol. 171, p. 104558, 2024.

[19] Y. Liu *et al.*, "Reachable-set model predictive path integral control for autonomous vehicles," *IEEE Robotics and Automation Letters*, vol. 10, no. 2, pp. 1234–1241, 2025.

[20] Anonymous, "Development and integration of control strategy for level-2 autonomous vehicle lane-keeping assist system." SpringerLink, 2024.

[21] J. Kim *et al.*, "Regenerative braking control strategy based on ai algorithm to improve driving comfort of autonomous vehicles," *Applied Sciences*, vol. 13, no. 2, 2023.

[22] S. Ahmadi *et al.*, "Real-time energy-optimal path planning for electric vehicles." arXiv, 2024.

[23] S. Liu *et al.*, "Edge computing for autonomous driving," *Proceedings of the IEEE*, vol. 107, no. 8, 2019.

[24] "Energy-efficient hybrid model predictive trajectory planning for autonomous electric vehicles." arXiv, 2024.

[25] M. S. Khan *et al.*, "Validation of safety of the intended functionality for autonomous vehicles," 2019.

[26] P. George *et al.*, "A comprehensive review of embedded systems in autonomous vehicles," 2024.

[27] M. Koschi *et al.*, "Scenario-based testing of automated driving systems," 2021.

[28] California DMV, "Autonomous vehicles testing with a driver," 2025.

[29] Anonymous, "Deep reinforcement learning for autonomous vehicles: Lane keep and overtaking scenarios," 2023.

[30] D. Prastiyanto *et al.*, "The adaptive cruise control for curved roads using archived crow search algorithm," *International Journal of Transport Development and Integration*, vol. 8, no. 1, pp. 179–188, 2024.

[31] B. Gao *et al.*, "Personalized adaptive cruise control based on online driving style recognition and model predictive control," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 11, pp. 12482–12496, 2020.